

UNIVERSIDAD TECNOLÓGICA NACIONAL.

Tecnicatura Universitaria en Programación.

Programación I.

Alumnos:

Braian Martinez.

Peralta Mateo.

INTRODUCCIÓN:	3
MARCO TEÓRICO:	3
ESTRUCTURA MODULAR DEL PROYECTO	6
METODOLOGÍA DE TRABAJO COLABORATIVO	6
DESARROLLO, MODULARIZACIÓN Y APORTES POR RAMAS	8
HISTORIAL DE VERSIONES Y REGISTRO	11
CAPTURAS DEL PROYECTO	13
ENLACES DEL PROYECTO	15
BIBLIOGRAFÍA:	16

INTRODUCCIÓN:

El presente informe detalla el diseño, desarrollo e implementación de una aplicación de consola en Python enfocada en la gestión y análisis estadístico de un conjunto de datos (dataset) de países.

El sistema fue concebido bajo los principios de la programación modular, la persistencia de datos mediante archivos de texto plano CSV y el desarrollo colaborativo utilizando Git y GitHub Desktop. El objetivo central del proyecto radica en la transición de un script monolítico hacia una arquitectura distribuida que garantice la alta disponibilidad del código, facilitando el mantenimiento y la escalabilidad del sistema ante futuras reglas de negocio

MARCO TEÓRICO:

El desarrollo de este sistema de gestión se fundamenta en los paradigmas de la programación estructurada y modular utilizando el lenguaje Python 3.x. A continuación, se detallan los conceptos aplicados y su relevancia en el diseño del software:

Listas:

En Python, las listas son colecciones de elementos indexadas, ordenadas y mutables. Permiten almacenar secuencias de datos dinámicamente y, en el contexto de este sistema, actúan como el contenedor principal para almacenar los registros de los países o las subcolecciones resultantes de los procesos de filtrado.

Ejemplo: `países = ["Argentina", "Brasil", "Japón"]`

Diccionarios:

Son estructuras de datos que almacenan pares de clave-valor, donde cada clave es única. A diferencia de las secuencias indexadas numéricamente, los diccionarios permiten un acceso semántico a la información. En este proyecto se utilizan para representar el dominio de cada país, asociando propiedades específicas como el nombre, población, superficie y continente a sus respectivos valores.

Ejemplo: `pais = {"nombre": "Argentina", "poblacion": 45376763}`

Funciones:

Representan bloques de código independientes que ejecutan una tarea específica al ser invocados. La modularización mediante funciones responde al principio de diseño de "una función = una responsabilidad", lo cual optimiza la legibilidad, facilita el mantenimiento del código, evita la duplicación y permite un testeado aislado de los componentes.

Ejemplo:

```
def calcular_promedio(total, cantidad):  
    return total / cantidad
```

Estructuras Condicionales:

Son instrucciones de control de flujo (if-elif-else) que evalúan expresiones booleanas para determinar el camino de ejecución del programa. Son indispensables para la toma de decisiones del menú de opciones, la validación de datos de entrada (evitando campos vacíos o tipos de datos incorrectos) y el control de rangos.

Ejemplo:

```
if superficie > 0:  
    print("Superficie válida")  
else:  
    print("Error: la superficie debe ser mayor a 0")
```

Estructuras Repetitivas:

Bucles (while y for) que permiten ejecutar un bloque de código de forma iterativa. El bucle while se emplea para mantener el ciclo de vida del menú en consola hasta que el usuario decida salir, mientras que el bucle for se utiliza para recorrer secuencialmente las estructuras de datos al realizar búsquedas, filtros u ordenamientos.

Ejemplo:

```
for pais in lista_paises:
```

Permite recorrer secuencialmente cada registro.

Algoritmos de Ordenamiento:

Son procedimientos lógicos que reorganizan los elementos de una lista siguiendo un criterio específico. El sistema aplica estos algoritmos sobre los datos de los países para ordenarlos de manera alfabética (por nombre) o numérica (por población o superficie), permitiendo una visualización tanto ascendente como descendente según la preferencia del operador.

Ejemplo:

Uso de funciones de ordenación (como ordenamiento por burbuja o métodos integrados) para clasificar por el valor numérico de la población.

Estadísticas Básicas:

Son funciones de agregación aplicadas a conjuntos de datos numéricos para la obtención de métricas e indicadores clave. El programa procesa las variables cuantitativas para

identificar valores extremos (máximos y mínimos en población y superficie), calcular la media aritmética (promedios generales) y determinar frecuencias (cantidad de países por continente).

Ejemplo:

Recorrer los datos para determinar qué país posee el menor número en su atributo de población.

Archivos CSV (Comma-Separated Values):

Es un formato estándar de texto plano utilizado para almacenar datos tabulares, donde cada línea representa un registro y las columnas se separan mediante un delimitador (habitualmente comas). Su uso permite la persistencia de los datos, logrando que el sistema sea capaz de leer la información base al iniciar y actualizarla sin necesidad de implementar sistemas complejos de bases de datos relacionales.

Ejemplo:

Lectura de registros usando el módulo nativo `open()` línea por línea o mediante la librería estándar `csv`.

ESTRUCTURA MODULAR DEL PROYECTO.

Cumpliendo con los requerimientos de diseño de software avanzados, el archivo histórico único de funciones fue segmentado en un paquete distribuido de dependencias jerárquicas. Cada componente responde de forma estricta al principio de responsabilidad única:

- [main.py](#):
Orquestador central del sistema. Gestiona el ciclo de vida de la aplicación mediante un bucle iterativo e interactúa directamente con el menú de usuario.
- [datos.py](#):
Centraliza el manejo de flujos de archivos I/O (lectura y escritura del repositorio persistente `países.csv`) y la renderización estética y tabulada de datos en consola.
- [validaciones.py](#):
Aloja las reglas de negocio y los controladores de mutación segura de datos.
- [busquedas.py](#):
Implementa los algoritmos de exploración y coincidencia parcial insensible a mayúsculas sobre los nombres de los países.
- [filtros.py](#):
Aísla las rutinas de discriminación y extracción sectorizada de datos por criterios de geografía o magnitudes numéricas.
- [ordenamiento.py](#):
Provee las herramientas de clasificación posicional basándose en claves lambda y argumentos booleanos variables, ordenando de forma ascendente y descendente.
- [estadisticas.py](#):
Módulo encargado exclusivamente del análisis agregativo y matemático de las variables cuantitativas del dataset.

METODOLOGÍA DE TRABAJO COLABORATIVO.

El desarrollo integral del proyecto se administró bajo un flujo de control de versiones distribuido utilizando Git y GitHub Desktop, garantizando la trazabilidad del trabajo en equipo exigido por la cátedra:

Definición de Requerimientos:

Se utilizó el sistema de incidentes de GitHub para auditar el código base. El integrante Braian Martinez documentó las debilidades del sistema en el Issue #1, estableciendo las pautas de optimización.

Aislamiento de Entornos:

El integrante Mateo Peralta creó y operó sobre la rama de desarrollo local mateo/correcciones, permitiendo codificar, probar y refactorizar las soluciones de robustez de manera aislada y sin alterar la estabilidad de la rama de producción.

Revisión de Código:

Finalizadas las correcciones, se inició una solicitud de extracción Pull Request, en el servidor remoto, adjuntando la documentación de los cambios para permitir el debate técnico y la auditoría de líneas de código entre los desarrolladores.

Integración Continua:

Tras validar la ausencia de regresiones y comprobar la correcta ejecución del sistema como módulo, se procedió a la fusión de la rama de características dentro de la rama principal, unificando el producto para su distribución final.

DESARROLLO, MODULARIZACIÓN Y APORTES POR RAMAS

El desarrollo del proyecto se ejecutó de forma secuencial y acumulativa, permitiendo que cada integrante aporte al sistema de manera organizada desde sus respectivas ramificaciones de código:

Configuración de Entorno y Arquitectura Base:

Rama: braian/gestor-paises.

Como inicio absoluto del proyecto, el integrante Mateo Peralta fue el encargado de inicializar el repositorio oficial en GitHub y configurar la rama principal "main" para establecer el entorno de trabajo remoto.

A partir de esa base limpia, el integrante Braian Martinez creó y trabajó sobre la rama "braian/gestor-paises" para resolver el desacoplamiento del script y sentar los cimientos del sistema. Los esfuerzos en esta ramificación se centraron en diseñar la estructura inicial de paquetes y asegurar la persistencia de los datos. Las soluciones aplicadas fueron:

- Diseño del Orquestador Central src/main.py:
Se estructuró el flujo principal del programa mediante un bucle infinito guiado por eventos de consola, menú interactivo, aislando la lógica de inicialización para cargar el dataset en memoria al arrancar.
- Abstracción de la Capa de Datos src/datos.py:
Se implementó la lógica de persistencia utilizando los métodos de la librería estándar csv, DictReader y DictWriter, logrando que el sistema sincronice los cambios automáticamente en el archivo físico "paises.csv" ante cualquier alta o modificación.
- Formateo Estético de Reportes por Consola:
Se diseñó el algoritmo de impresión tabular dentro del módulo de datos, aplicando máquinas de formato de cadenas string para garantizar que las listas y estadísticas se rendericen de manera limpia y legible para el operador.

Auditoría e Ingeniería de Robustez:

Rama: mateo/correcciones:

Una vez que la estructura base fue consolidada en el repositorio, el integrante Mateo Peralta creó la rama local "mateo/correcciones" como un clon directo del trabajo de Braian para iniciar una etapa de auditoría, control de calidad QA y pruebas funcionales. En esta segunda fase, se detectaron y solucionaron áreas críticas asociadas a la validación de datos de entrada y la tolerancia a fallos:

- Robustez en Alta de Registros src/validaciones.py:

Se integró una función interna de normalización de caracteres que mapea las vocales con acentuación gráfica a sus equivalentes planas y convierte el flujo a minúsculas para la comparación contra una lista cerrada.

Por ejemplo, rechazando o aceptando de igual forma "america del sur" o "ÁFRICA". Una vez validado, el sistema recupera el formato estético original para su almacenamiento.

- **Restricción de Tipo Simbólico en Nombres:**
Se bloqueó la inserción de cadenas puramente numéricas en el identificador del país utilizando la función integrada `.isdigit()`, impidiendo anomalías lógicas en los registros. Por ejemplo, un país denominado "12345".
- **Control de Signo en Variables Cuantitativas:**
Se incorporaron estructuras condicionales estrictas para evaluar que las entradas de población y superficie sean estrictamente mayores a cero, impidiendo valores negativos o nulos.

Flexibilidad y Sanitización de Entradas Numéricas `src/datos.py` y `src/validaciones.py`:

Tratamiento de Separadores de Miles en Modificaciones:

El usuario final tiende a ingresar números grandes utilizando puntos de manera instintiva. Ejemplo, 46.285.124.

La conversión directa a tipo entero arrojaba una excepción fatal `ValueError`, interrumpiendo la ejecución del programa. Se implementó una limpieza elástica de strings mediante el método `.replace(".", "")` dentro de la función `actualizar_pais` previo al casteo.

Estandarización del Tipo de Dato de Superficie:

Se modificó el esquema de los diccionarios para almacenar el atributo superficie de manera estricta como un tipo entero (`int`) en lugar de punto flotante (`float`), dado que las extensiones territoriales provistas por el dataset base no requerían de precisión decimal, optimizando el uso de memoria en las operaciones internas.

Filtros Avanzados Tolerantes a Errores `src/filtros.py`:

Sanitización de Puntos de Miles en Filtros Cuantitativos:

Las búsquedas por rangos fallaban si el operador especificaba los límites numéricos acompañados de puntos. Ejemplo, un rango de población entre 10.000.000 y 50.000.000, o rangos de superficie con formato similar.

Se extendió la lógica de sanitización elástica (`.replace(".", "")`) a las funciones `filtrar_poblacion` y `filtrar_superficie`, aislando los caracteres de puntuación antes de evaluar las expresiones booleanas de rango.

Ciclo de Integración y Cierre del Proyecto:

Finalizadas y testeadas de forma local todas las mejoras de robustez en la rama "mateo", se procedió a realizar una solicitud de extracción, Pull Request, hacia el servidor remoto. El integrante Braian Martínez auditó el código propuesto, validó que cumpliera con los criterios de aceptación y aprobó formalmente la solicitud en la plataforma de GitHub. Finalmente, Braian ejecutó el Merge definitivo, unificando los cambios de manera exitosa en la rama principal "main" y cerrando el ciclo de desarrollo colaborativo.

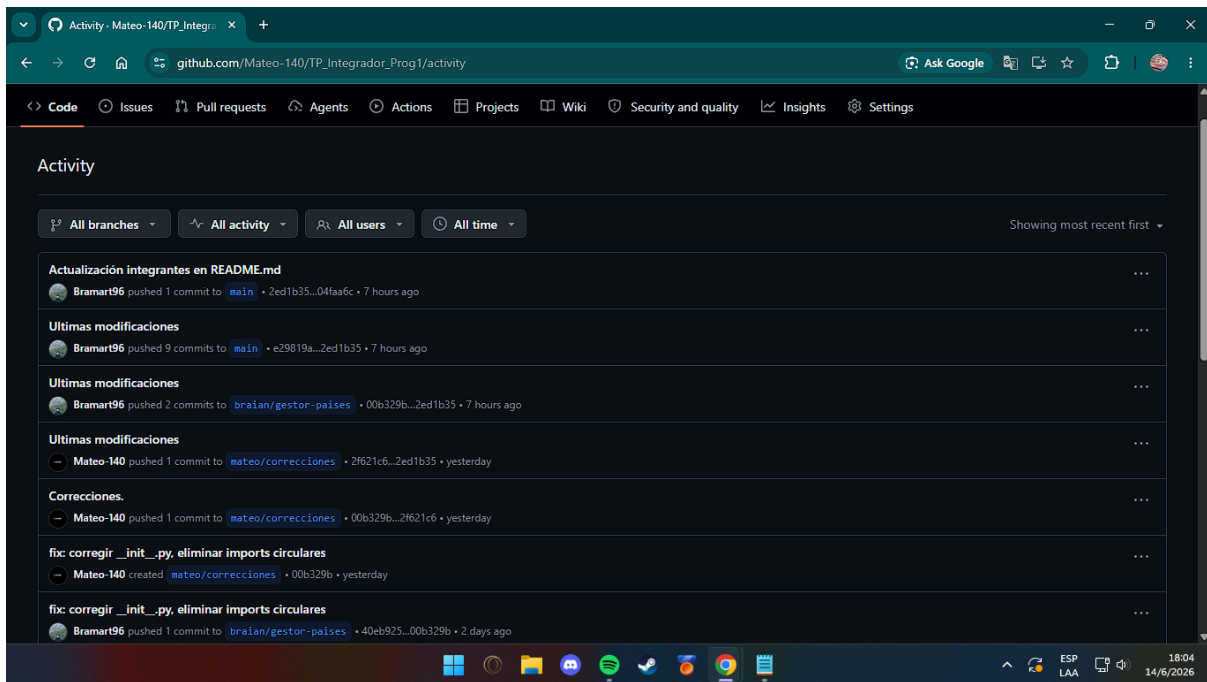
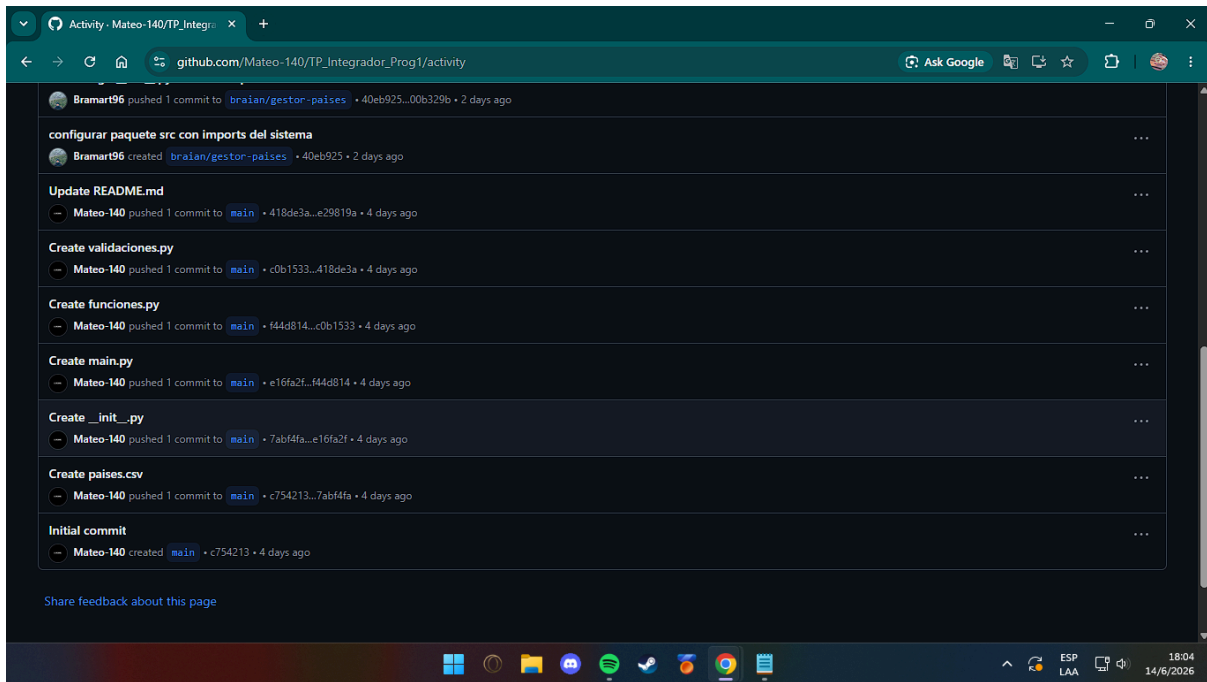
HISTORIAL DE VERSIONES Y REGISTRO.

Rama de Origen	Autor	Descripción Técnica del Commit / Acción
main.	Mateo Peralta.	Initial commit. Estructuración inicial del repositorio.
main.	Mateo Peralta.	Create paises.csv. Carga del dataset base.
main.	Mateo Peralta.	Create __init__.py. Inicialización de espacio de nombres.
main.	Mateo Peralta.	Create main.py. Orquestador y bucle principal.
main.	Mateo Peralta.	Create funciones.py . Script monolítico funcional.
main.	Mateo Peralta.	Create validaciones.py. Validaciones del sistema.
main.	Mateo Peralta.	Update README.md. Documentación técnica de inicio.
braian/gestor-paises.	Braian Martinez.	Configurar paquete src con imports del sistema.
braian/gestor-paises.	Braian Martinez.	fix: corregir __init__.py. Eliminar imports circulares.
mateo/correcciones.	Mateo Peralta.	fix: corregir __init__.py. Eliminar imports circulares.
mateo/correcciones.	Mateo Peralta.	Correcciones. Implementación de robustez numérica.
mateo/correcciones.	Mateo Peralta.	Últimas modificaciones. Sanitización elástica de puntos.

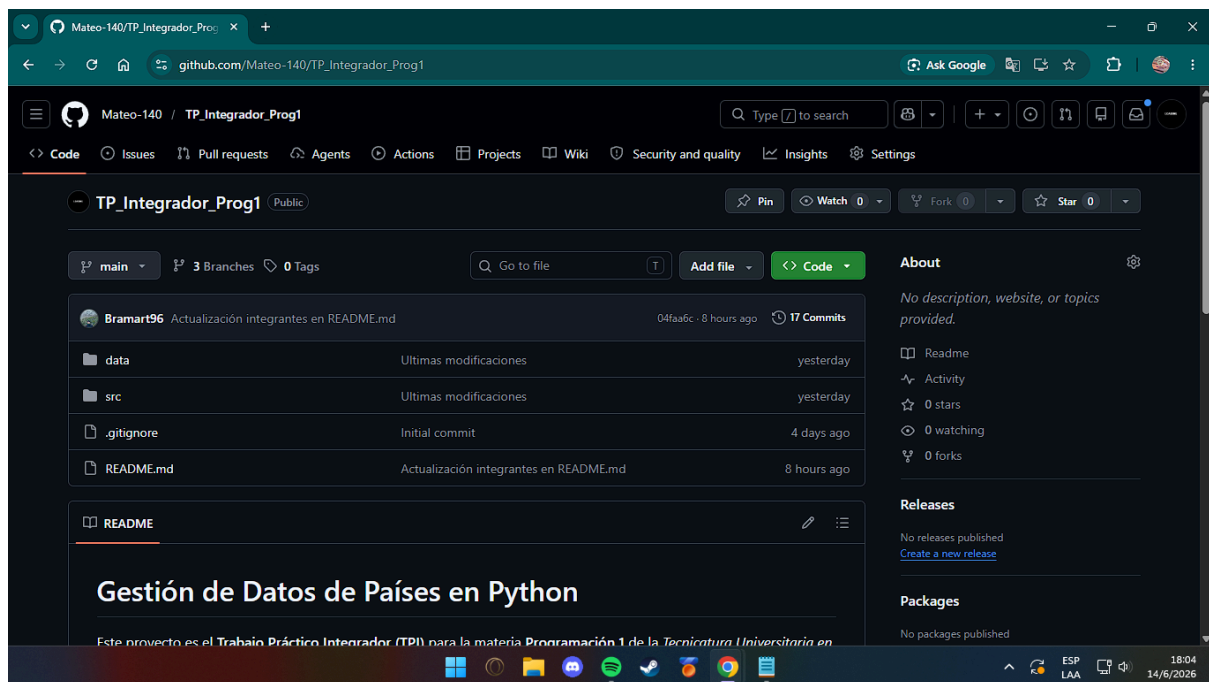
braian/gestor-paises.	Braian Martinez.	Últimas modificaciones. Ajustes de persistencia I/O.
main.	Braian Martinez.	Últimas modificaciones Merge general a rama estable.
main.	Braian Martinez.	Actualización de integrantes en README.md . Cierre del TPI.

CAPTURAS DEL PROYECTO.

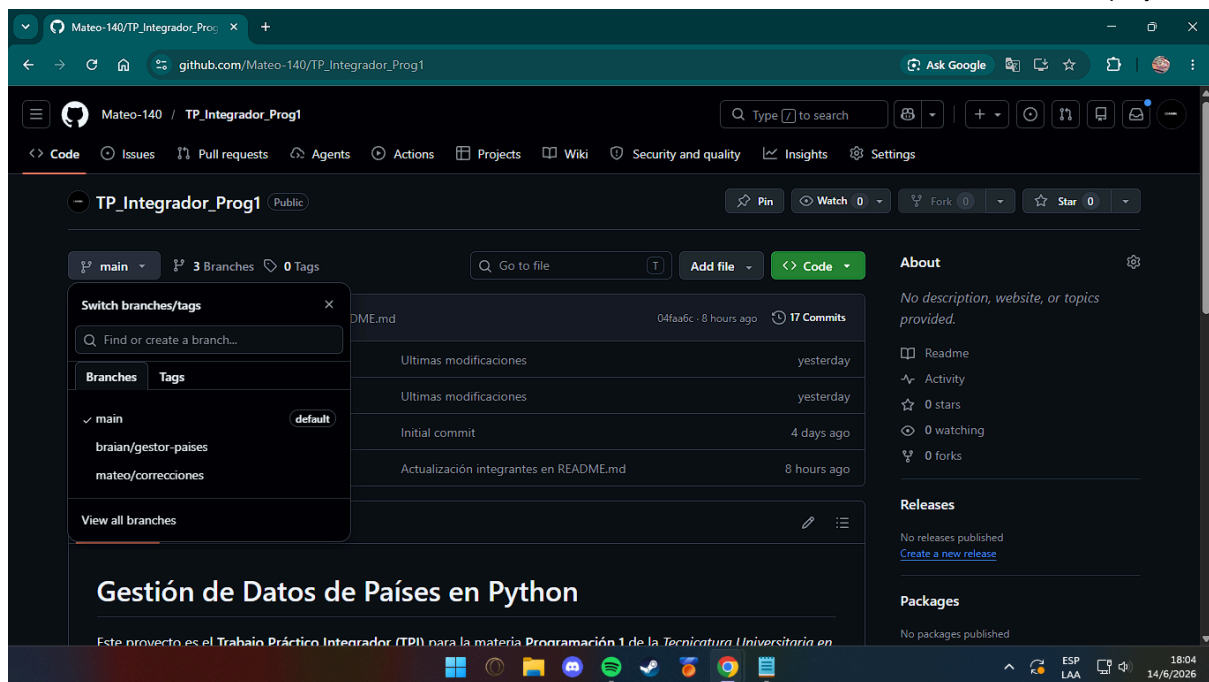
Historial de commits del proyecto dentro de GitHub.



Rama principal "main".



Ramas del proyecto.



The screenshot shows a GitHub issue page for 'Mejoras identificadas durante pruebas funcionales #1' in the repository 'Mateo-140/TP_Integrador_Prog1'. The issue was opened 2 days ago by Bramart96. The description lists several improvements identified during functional tests, such as validating country names, normalizing titles, and improving error messages. It also mentions that these improvements will be implemented in separate branches for traceability. The right sidebar shows the issue's metadata, including assignees (none), labels (none), projects (none), milestones (none), relationships (none), and development status (none yet). A 'Create sub-issue' button is visible below the description. The bottom of the page shows a Windows taskbar with various application icons and a system clock indicating 20:01 on 14/6/2026.

Mejoras identificadas durante p x +

github.com/Mateo-140/TP_Integrador_Prog1/issues/1

Ask Google

Bramart96 opened 2 days ago Collaborator

Durante la ejecución de pruebas funcionales se identificaron oportunidades de mejora para aumentar la robustez del sistema:

- Validar nombres de países numéricos.
- Normalizar nombres utilizando formato título.
- Mejorar mensajes de error para el usuario.
- Permitir puntos como separadores de miles en actualización de población.
- Validar continentes contra una lista permitida.
- Ignorar tildes y diferencias de mayúsculas/minúsculas en filtros por continente.
- Almacenar superficie como entero para mantener consistencia en el CSV.

Estas mejoras serán implementadas en ramas separadas para mantener trazabilidad y facilitar la revisión antes del merge final.

Create sub-issue

Mateo-140 yesterday Owner

Se tomaron las observaciones sobre las vulnerabilidades del sistema y procedí a realizar las correcciones necesarias en mi rama local mateo.

Se modificaron los módulos de alta y actualización para dotar al software de mayor robustez, se implementó la reestructuración completa del código histórico en 7 archivos independientes según los principios de la programación modular, y se dejó el código 100% funcional y testeado desde la consola. Quedo a la espera de tu revisión en el Pull Request para unificar las ramas de manera colaborativa.

Assignees: No one - [Assign yourself](#)

Labels: No labels

Projects: No projects

Milestone: No milestone

Relationships: None yet

Development: [Modularización del sistema y mejoras de robustez](#) (Mateo-140/TP_Integrador_Prog1)

Notifications: [Unsubscribe](#)

You're receiving notifications because you're subscribed to this thread.

20:01 14/6/2026

The screenshot shows the same GitHub issue page, but now with a history of updates and a comment section. The issue is marked as 'Closed'. The history shows that Mateo-140 closed the issue as 'completed' yesterday, reopened it yesterday, and mentioned it yesterday. Bramart96 closed the issue as 'completed' in 2 hours ago. The right sidebar shows the issue's metadata, including relationships (none yet), development status (none yet), and a list of actions: Transfer issue, Clone issue, Lock conversation, Pin issue, Delete issue, and Give feedback. The bottom of the page shows a Windows taskbar with various application icons and a system clock indicating 20:01 on 14/6/2026.

Mejoras identificadas durante pruebas funcionales #1

History:

- Mateo-140 closed this as **completed** yesterday
- Mateo-140 reopened this yesterday
- Mateo-140 mentioned this yesterday
- [Modularización del sistema y mejoras de robustez](#) #2
- Bramart96 closed this as **completed** in #2 9 hours ago

Add a comment

Write Preview

Use Markdown to format your comment

Paste, drop, or click to add files

Reopen issue

Comment

Relationships: None yet

Development: [Modularización del sistema y mejoras de robustez](#) (Mateo-140/TP_Integrador_Prog1)

Notifications: [Unsubscribe](#)

You're receiving notifications because you're subscribed to this thread.

Participants:

- [Transfer issue](#)
- [Clone issue](#)
- [Lock conversation](#)
- [Pin issue](#)
- [Delete issue](#)
- [Give feedback](#)

20:01 14/6/2026

ENLACES DEL PROYECTO.

Repositorio GitHub:

https://github.com/Mateo-140/TP_Integrador_Prog1

Video Demostrativo del Funcionamiento:

<https://youtu.be/qe8kJ0Rlx-E>

BIBLIOGRAFÍA:

Para el diseño, fundamentación y desarrollo del presente proyecto, se utilizaron los siguientes recursos y fuentes oficiales como respaldo técnico:

Python Software Foundation (2026). Python 3.x Documentation: Data Structures, Built-in Functions and File I/O.

<https://docs.python.org/3/>

Van Rossum, G., Warsaw, B., & Coghlan, N. (2013). PEP 8 – Style Guide for Python Code. <https://peps.python.org/pep-0008/>Universidad Tecnológica Nacional (2026).

Material de estudio de la materia Programación 1 - Tecnicatura Universitaria en Programación. UTN.